



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА №1

«Основные конструкции языка Python»

**ДИСЦИПЛИНА: «Парадигмы и конструкция языков
программирования»**

Выполнил: студент гр. ИУ5-34Б

_____ (Бондаренко Д.А.)
(Подпись) (Ф.И.О.)

Проверил:

_____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

1. Описание задания

Цель работы: изучение основных конструкций языка Python.

Задание: разработать программы для решения биквадратного уравнения.

Программы должны быть разработаны в виде консольного приложения на языке Python. Требуется разработать три программы: с применением процедурной парадигмы (использование функций, оператор def); с применением объектно-ориентированной парадигмы (использование классов); с применением функциональной парадигмы (сопоставление с образцом).

Каждая программа осуществляет ввод с клавиатуры коэффициентов A , B , C , вычисляет дискриминант и действительные корни уравнения (в зависимости от дискриминанта).

Коэффициенты A , B , C могут быть заданы в виде параметров командной строки. Если они не заданы — вводятся с клавиатуры. Если коэффициент введен или задан в командной строке некорректно, необходимо проигнорировать некорректное значение и вводить коэффициент повторно, пока он не будет введен корректно. Корректно заданный коэффициент — это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.

Математическая основа. Биквадратное уравнение $ax^4 + bx^2 + c = 0$ решается заменой $t = x^2$. Получается квадратное уравнение $at^2 + bt + c = 0$. Находится дискриминант $D = b^2 - 4ac$. В зависимости от D находятся t , затем для каждого $t \geq 0$ вычисляются $x = \pm\sqrt{t}$. Дубликаты удаляются.

2. Текст программы

2.1. Процедурная парадигма — файл bikvadr_proc.py

```
import sys
import math
def get_coef(index, prompt):
    while True:
        try:
            coef_str = sys.argv[index]
        except IndexError:
            print(prompt)
            coef_str = input()
        try:
            return float(coef_str)
        except ValueError:
            print('Невозможно преобразовать "{}" в число. Повторите ввод.'.format(coef_str))
            if index < len(sys.argv):
```

```
sys.argv.pop(index)
```

```
def get_roots(a, b, c):
    result=[]
    D=b*b-4*a*c
    if D<0.0:
        return result
    if D==0.0:
        t_list=[-b/(2.0*a)]
    else:
        sqD=math.sqrt(D)
        t_list=[(-b+sqD)/(2.0*a),
                (-b-sqD)/(2.0*a)]
    for t in t_list:
        if t>0.0:
            sq=math.sqrt(t)
            result.append(sq)
            result.append(-sq)
        elif t==0.0:
            result.append(0.0)
    unique=[]
    for r in result:
        if r not in unique:
            unique.append(r)
    return unique

def main():
    a=get_coef(1,'Введите число a:')
    b=get_coef(2,'Введите число b:')
    c=get_coef(3,'Введите число c:')
    roots=get_roots(a,b,c)
    len_roots=len(roots)
    if len_roots==0:
        print('Корней нет')
    elif len_roots==1:
        print('Один корень: {}".format(roots[0]))
    elif len_roots==2:
        print('Два корня: {}, {}".format(roots[0],roots[1]))
    elif len_roots==3:
        print('Три корня: {}, {}, {}".format(roots[0],roots[1],roots[2]))
    else:
        print('Четыре корня: {}, {}, {}, {}'.format(roots[0],roots[1],roots[2],roots[3]))

if __name__=="__main__":
    main()
```

2.2. Объектно-ориентированная парадигма — файл bikvadr_oop.py

```
import sys
import math
class BiquadraticRoots:
    def __init__(self):
        self.coef_a=0.0
        self.coef_b=0.0
        self.coef_c=0.0
        self.num_roots=0.0
        self.roots_list=[]
    def get_coef(self,index,prompt):
        while True:
            try:
                coef_str=sys.argv[index]
            except IndexError:
                print(prompt)
                coef_str=input()
            try:
                return float(coef_str)
            except ValueError:
                print('Невозможно преобразовать "{}" в число.Повторите
ввод.'.format(coef_str))
                if index < len(sys.argv):
                    sys.argv.pop(index)
    def get_coefs(self):
        self.coef_a=self.get_coef(1,'Введите число a:')
        self.coef_b=self.get_coef(2,'Введите число b:')
        self.coef_c=self.get_coef(3,'Введите число c:')
    def calculate_roots(self):
        a=self.coef_a
        b=self.coef_b
        c=self.coef_c
        D=b*b-4*a*c
        if D<0.0:
            t_list=[]
        elif D==0.0:
            t_list=[-b/(2.0*a)]
        else:
            sqD=math.sqrt(D)
            t_list=[(-b+sqD)/(2.0*a),
                    (-b-sqD)/(2.0*a)]
        for t in t_list:
```

```

    if t>0.0:
        sq=math.sqrt(t)
        self.roots_list.append(sq)
        self.roots_list.append(-sq)
    elif t==0.0:
        self.roots_list.append(0.0)
unique=[]
for r in self.roots_list:
    if r not in unique:
        unique.append(r)
self.roots_list=unique
self.num_roots=len(self.roots_list)
def print_roots(self):
    if self.num_roots==0:
        print('Корней нет')
    elif self.num_roots==1:
        print('Один корень: {}".format(self.roots_list[0]))
    elif self.num_roots==2:
        print('Два корня: {}, {}".format(self.roots_list[0],self.roots_list[1]))
    elif self.num_roots==3:
        print('Три корня: {}, {},
"{}".format(self.roots_list[0],[1],self.roots_list[2]))
    else:
        print('Четыре корня: {}, {}, {},
"{}".format(self.roots_list[0],self.roots_list[1],self.roots_list[2],self.roots_list[3
]))
def main():
    r=BiquadraticRoots()
    r.get_coefs()
    r.calculate_roots()
    r.print_roots()
if __name__=="__main__":
    main()

```

2.3. Функциональная парадигма — файл bikvadr_namedtuple.py Python

```

import sys
import math
from typing import NamedTuple
class BiquadraticResult:
    NoRoots=NamedTuple("NoRoots",[])
    OneRoot=NamedTuple("OneRoot", [("root",float)])
    TwoRoots=NamedTuple("TwoRoots", [("root1",float),
        ("root2",float)])
    ThreeRoots=NamedTuple("ThreeRoots", [("root1",float),

```

```

        ("root2", float),
        ("root3", float)])
FourRoots=NamedTuple("FourRoots", [("root1", float),
        ("root2", float),
        ("root3", float),
        ("root4", float)])
def get_coef(index, prompt):
    while True:
        try:
            coef_str=sys.argv[index]
        except IndexError:
            print(prompt)
            coef_str=input()
        try:
            return float(coef_str)
        except ValueError:
            print('Невозможно преобразовать "{}" в число.Повторите
ввод.'.format(coef_str))
            if index < len(sys.argv):
                sys.argv.pop(index)

def get_roots(a,b,c):
    D=b*b-4*a*c
    if D<0.0:
        t_list=[]
    elif D==0.0:
        t_list=[-b/(2.0*a)]
    else:
        sqD=math.sqrt(D)
        t_list=[(-b+sqD)/(2.0*a),
                (-b-sqD)/(2.0*a)]
    roots=[]
    for t in t_list:
        if t>0.0:
            sq=math.sqrt(t)
            roots.append(sq)
            roots.append(-sq)
        elif t==0.0:
            roots.append(0.0)
    unique=[]
    for r in roots:
        if r not in unique:
            unique.append(r)
    match unique:
        case[]:

```

```

        return BiquadraticResult.NoRoots()
    case [r]:
        return BiquadraticResult.OneRoot(r)
    case [r1,r2]:
        return BiquadraticResult.TwoRoots(r1,r2)
    case [r1,r2,r3]:
        return BiquadraticResult.ThreeRoots(r1,r2,r3)
    case [r1,r2,r3,r4]:
        return BiquadraticResult.FourRoots(r1,r2,r3,r4)
def print_roots(result):
    match result:
        case BiquadraticResult.NoRoots():
            print('Корней нет')
        case BiquadraticResult.OneRoot(root):
            print('Один корень: {}".format(root))
        case BiquadraticResult.TwoRoots(root1,root2):
            print('Два корня: {},{}".format(root1,root2))
        case BiquadraticResult.ThreeRoots(root1,root2,root3):
            print('Три корня: {},{},{}".format(root1,root2,root3))
        case BiquadraticResult.FourRoots(root1,root2,root3,root4):
            print('Четыре корня: {},{},{},{}".format(root1,root2,root3,root4))
def main():
    a=get_coef(1,'Введите число a:')
    b=get_coef(2,'Введите число b:')
    c=get_coef(3,'Введите число c:')
    result=get_roots(a,b,c)
    print_roots(result)

if __name__=="__main__":
    main()

```

3. Экранные формы с примерами выполнения программы

3.1. Программа 1 (bikvadr_proc.py)

Запуск с двумя корнями:

```

danil@MacBook-Air-Danil lr1 % python3 bikvadr_proc.py 1 0 -4
Два корня: "1.4142135623730951", "-1.4142135623730951"

```

Запуск с четырьмя корнями:

```

danil@MacBook-Air-Danil lr1 % python3 bikvadr_proc.py 1 -5 4
Четыре корня: "2.0", "-2.0", "1.0", "-1.0"

```

Запуск без корней:

```

danil@MacBook-Air-Danil lr1 % python3 bikvadr_proc.py 1 0 4
Корней нет

```

Запуск с одним корнем:

```

danil@MacBook-Air-Danil lr1 % python3 bikvadr_proc.py 1 0 0
Один корень: "0.0"

```

Запуск с клавиатуры (без аргументов):

```
Введите число a:
1
Введите число b:
0
Введите число c:
-4
Два корня: "1.4142135623730951", "-1.4142135623730951"
```

Запуск с некорректным аргументом:

```
Введите число a:
abc
Невозможно преобразовать "abc" в число.Повторите ввод.
Введите число a:
1
Введите число b:
-5
Введите число c:
4
Четыре корня:"2.0","-2.0","1.0", "-1.0"
```

3.2. Программа 2 (bikvadr_oop.py)

Запуск с четырьмя корнями:

```
danil@MacBook-Air-Danil lr1 % python3 bikvadr_oop.py 1 0 -4
Два корня: "1.4142135623730951", "-1.4142135623730951"
```

Запуск без корней:

```
danil@MacBook-Air-Danil lr1 % python3 bikvadr_oop.py 1 0 4
Корней нет
```

Запуск с некорректным аргументом:

```
Введите число a:
хуз
Невозможно преобразовать "хуз" в число.Повторите ввод.
Введите число a:
1
Введите число b:
-5
Введите число c:
4
Четыре корня:"2.0","-2.0","1.0", "-1.0"
```

3.3. Программа 3 (bikvadr_namedtuple.py)

Запуск с двумя корнями:

```
danil@MacBook-Air-Danil lr1 % python3 bikvadr_namedtuple.py 1 0 -4
Два корня: "1.4142135623730951", "-1.4142135623730951"
```

Запуск с четырьмя корнями:

```
danil@MacBook-Air-Danil lr1 % python3 bikvadr_namedtuple.py 1 -5 4
Четыре корня: "2.0","-2.0","1.0", "-1.0"
```

Запуск с одним корнем:

```
danil@MacBook-Air-Danil lr1 % python3 bikvadr_namedtuple.py 1 0 0
Один корень: "0.0"
```

Запуск без корней:

```
danil@MacBook-Air-Danil lr1 % python3 bikvadr_namedtuple.py 1 0 4
Корней нет
```

Вывод:

В ходе выполнения лабораторной работы №1 были изучены основные конструкции языка Python и разработаны три программы для решения биквадратного уравнения $ax^4 + bx^2 + c = 0$.

Программы реализованы в трёх парадигмах программирования: процедурной (с использованием функций и оператора def), объектно-ориентированной (с использованием классов, полей и методов) и функциональной (с использованием NamedTuple и сопоставления с образцом match/case).

В ходе работы были освоены следующие темы: работа с аргументами командной строки через sys.argv, подключение модуля math и использование функции math.sqrt, обработка ошибок через try/except (IndexError, ValueError), организация повторного ввода через цикл while True, удаление элементов списка через pop(), работа с классами (__init__, self, поля объекта), а также использование NamedTuple и match/case для функционального подхода.

Все три программы дают одинаковый результат на одинаковых входных данных. Это подтверждает, что разные парадигмы могут решать одну и ту же задачу, различаясь лишь способом организации кода.